

PORTFOLIO PROJECT DOCUMENTATION

Serverless Image Processing & Metadata Pipeline on AWS Lambda

Event-driven architecture built with Lambda, S3, DynamoDB, SNS/SQS,
provisioned with Terraform and deployed through a GitHub Actions CI/CD pipeline

Prepared by **Mustapha**
Aspiring DevOps / Cloud Engineer
GitHub: github.com/mustapha641
September 2026

Contents

1. Why I Built This
2. System Context (Level 1)
3. Component Architecture (Level 2)
4. Request Flow / Sequence Diagram (Level 3)
5. Infrastructure as Code (Terraform)
6. Application Code (Lambda Handler + Unit Tests)
7. CI/CD Pipeline (Level 4)
8. Security & IAM Design
9. Observability & Monitoring
10. Cost Considerations
11. Problems I Ran Into (and how I fixed them)
12. What I Tested / Results
13. Skills Map (for reviewers)
14. What I'd Add Next
15. Repository Layout

1. Why I Built This

Most of my earlier hands-on work was around provisioning long-running infrastructure — VPCs, EC2 behind a load balancer, RDS in a private subnet, all wired together with Terraform. That taught me a lot about networking and IaC, but it didn't touch the other half of what a lot of DevOps job postings actually ask for: event-driven automation, serverless compute, and the operational side of things like dead-letter queues, alarms, and cost control on pay-per-use services.

So I picked a problem that's small enough to finish solo but still forces you to touch every part of the AWS "serverless glue" toolkit: a pipeline that takes an image dropped into an S3 bucket, resizes it, pulls out its metadata (dimensions, format, EXIF where available), stores that metadata in DynamoDB, and notifies a subscriber when it's done — with proper retry handling, monitoring, and a CI/CD pipeline instead of clicking around the AWS console.

The goal wasn't to build something novel. It was to build something a hiring manager could open, read the Terraform, read the Lambda code, look at the pipeline run history, and come away confident that I understand how these pieces fit together in production, not just in a tutorial.

Objectives

- No manual console clicks — everything reproducible from `terraform apply`.
- Handle failure paths explicitly (retries, DLQ, alerting) rather than just the happy path.
- Keep IAM permissions scoped to exactly what each function needs.
- Automate testing and deployment through a pipeline, with a manual approval gate before touching real infrastructure.
- Make the whole thing cheap enough to run indefinitely on the AWS free tier for demo purposes.

2. System Context (Level 1)

At the highest level, there's a client uploading an image, an AWS-hosted pipeline that does the work, and two outcomes: the uploader (or a downstream system) gets notified the image is ready, and the operations side gets alerted if something breaks.

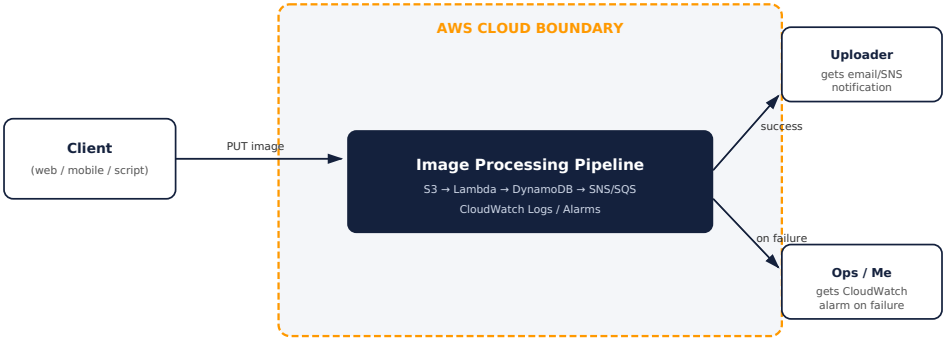


Fig 1. System context — a black-box view of the pipeline and its two outside actors.

This is deliberately the "explain it to a non-engineer" view. Nobody outside the boundary needs to know or care whether it's Lambda, Fargate, or a fleet of EC2 boxes doing the work — that decision lives inside the boundary, and I made it below.

3. Component Architecture (Level 2)

Inside the boundary, here's what's actually running. I split processing and notification into two separate Lambda functions rather than one monolithic function — partly for the single-responsibility reason, but mostly because it let me give each function its own, narrower IAM role and its own concurrency/timeout tuning.

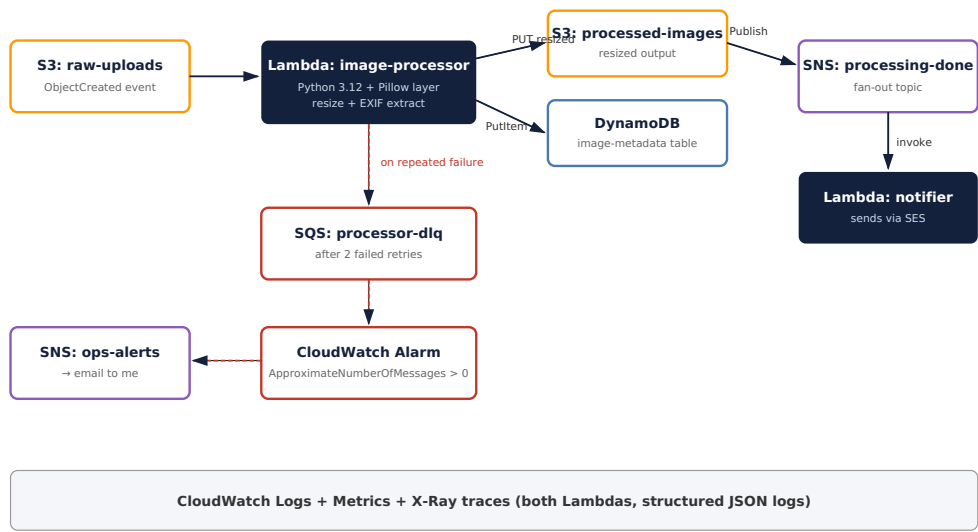


Fig 2. Component diagram — the failure path (red, dashed) is drawn deliberately alongside the happy path (black), not as an afterthought.

Component notes

Component	Purpose	Key config decision
S3 raw-uploads	Landing zone for client uploads	Versioning off, lifecycle rule expires objects after 7 days — it's a staging bucket, not storage
Lambda image-processor	Resize + extract EXIF/dimensions/format	512 MB memory (gives ~2 vCPU worth of compute per AWS's memory→CPU scaling), 30s timeout, reserved concurrency of 5
S3 processed-images	Stores the resized output	Server-side encryption (SSE-S3) enabled by default
DynamoDB image-metadata	One item per processed image: dimensions, format, size, timestamp	On-demand billing mode — traffic is spiky and low-volume, provisioned capacity would be wasted spend
SNS processing-done	Decouples "processing finished" from "who needs to know"	Lets me add more subscribers later (Slack webhook, another queue) without touching the processor
Lambda notifier	Sends the actual email via SES	128 MB memory — it does almost nothing, no reason to pay for more
SQS processor-dlq	Catches events the processor failed to handle after retries	14-day retention so I have time to investigate before it's gone
CloudWatch Alarm + ops-alerts	Tells me (not the end user) something broke	Threshold of 1 message in the DLQ — I'd rather get paged too often on a side project than miss a real failure

This is the view I actually used while debugging — it maps directly onto what you see reading CloudWatch Logs Insights output line by line.



One detail worth calling out because it tripped me up initially: `S3 → Lambda` is an **asynchronous** invocation. That means Lambda's own retry policy (not S3's) governs what happens on failure — by default two retries with a backoff, and only after those are exhausted does the event land in the DLQ I configured. I originally assumed S3 would keep re-firing the event, which is not how it works and cost me a debugging session before I read the invocation-type docs properly.

5. Infrastructure as Code (Terraform)

Everything above is provisioned through Terraform, split into modules (`s3`, `lambda`, `dynamodb`, `messaging`, `iam`) with a root module that wires them together per environment (`dev/prod` workspaces). Below are the pieces that mattered most to get right.

5.1 Lambda function + S3 trigger

modules/lambda/main.tf

```
resource "aws_lambda_function" "image_processor" {
  function_name = "${var.project}-image-processor-${var.env}"
  filename      = data.archive_file.processor_zip.output_path
  source_code_hash = data.archive_file.processor_zip.output_base64sha256
  handler       = "handler.lambda_handler"
  runtime       = "python3.12"
  role          = aws_iam_role.processor_role.arn
  memory_size   = 512
  timeout       = 30
  reserved_concurrent_executions = 5
  layers        = [aws_lambda_layer_version.pillow_layer.arn]

  environment {
    variables = {
      PROCESSED_BUCKET = var.processed_bucket_name
      METADATA_TABLE   = var.metadata_table_name
      SNS_TOPIC_ARN     = var.sns_topic_arn
      LOG_LEVEL        = var.log_level
    }
  }

  dead_letter_config {
    target_arn = aws_sqs_queue.processor_dlq.arn
  }

  tracing_config {
    mode = "Active" # X-Ray
  }
}

resource "aws_s3_bucket_notification" "raw_upload_trigger" {
  bucket = var.raw_bucket_id

  lambda_function {
    lambda_function_arn = aws_lambda_function.image_processor.arn
    events              = ["s3:ObjectCreated:*"]
    filter_suffix       = ".jpg" # also mirrored for .jpeg / .png in a second block
  }
}

resource "aws_lambda_permission" "allow_s3_invoke" {
  statement_id = "AllowS3Invoke"
  action       = "lambda:InvokeFunction"
  function_name = aws_lambda_function.image_processor.function_name
  principal    = "s3.amazonaws.com"
  source_arn    = var.raw_bucket_arn
}
```

5.2 Least-privilege IAM policy

modules/iam/processor_policy.tf

```
data "aws_iam_policy_document" "processor_permissions" {
  statement {
    sid = "ReadRawBucket"
    actions = ["s3:GetObject"]
    resources = ["${var.raw_bucket_arn}/*"]
  }

  statement {
    sid = "WriteProcessedBucket"
    actions = ["s3:PutObject"]
    resources = ["${var.processed_bucket_arn}/*"]
  }

  statement {
    sid = "WriteMetadataTable"
    actions = ["dynamodb:PutItem"]
    resources = [var.metadata_table_arn]
  }

  statement {
    sid = "PublishStatus"
    actions = ["sns:Publish"]
    resources = [var.sns_topic_arn]
  }
}
```

```

}

statement {
  sid      = "WriteLogs"
  actions  = [
    "logs:CreateLogGroup",
    "logs:CreateLogStream",
    "logs:PutLogEvents"
  ]
  resources = ["arn:aws:logs:${var.region}:${var.account_id}:log-group:/aws/lambda/**"]
}
}
}

```

Every statement is scoped to a specific resource ARN, not `"*"`. No statement grants both read and write on the same bucket to the same function unless that function actually needs both.

5.3 DynamoDB table

modules/dynamodb/main.tf

```

resource "aws_dynamodb_table" "image_metadata" {
  name           = "${var.project}-image-metadata-${var.env}"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "image_id"

  attribute {
    name = "image_id"
    type = "S"
  }

  point_in_time_recovery {
    enabled = true
  }

  server_side_encryption {
    enabled = true
  }
}

```

6. Application Code (Lambda Handler + Unit Tests)

6.1 Handler

src/processor/handler.py

```
import json
import logging
import os
import uuid
from datetime import datetime, timezone
from io import BytesIO

import boto3
from PIL import Image, ExifTags

logger = logging.getLogger()
logger.setLevel(os.environ.get("LOG_LEVEL", "INFO"))

s3 = boto3.client("s3")
dynamodb = boto3.resource("dynamodb")
sns = boto3.client("sns")

PROCESSED_BUCKET = os.environ["PROCESSED_BUCKET"]
METADATA_TABLE = os.environ["METADATA_TABLE"]
SNS_TOPIC_ARN = os.environ["SNS_TOPIC_ARN"]
THUMBNAIL_SIZE = (800, 800)

def lambda_handler(event, context):
    """Entry point invoked asynchronously by an S3 ObjectCreated event."""
    table = dynamodb.Table(METADATA_TABLE)

    for record in event["Records"]:
        source_bucket = record["s3"]["bucket"]["name"]
        source_key = record["s3"]["object"]["key"]
        image_id = str(uuid.uuid4())

        logger.info(json.dumps({
            "event": "processing_start",
            "image_id": image_id,
            "source_key": source_key,
        }))

        try:
            raw_bytes = _download(source_bucket, source_key)
            resized_bytes, metadata = _resize_and_extract(raw_bytes)
            processed_key = f"processed/{image_id}.jpg"

            s3.put_object(
                Bucket=PROCESSED_BUCKET,
                Key=processed_key,
                Body=resized_bytes,
                ContentType="image/jpeg",
            )

            table.put_item(Item={
                "image_id": image_id,
                "source_key": source_key,
                "processed_key": processed_key,
                "width": metadata["width"],
                "height": metadata["height"],
                "format": metadata["format"],
                "size_bytes": len(resized_bytes),
                "processed_at": datetime.now(timezone.utc).isoformat(),
            })

            sns.publish(
                TopicArn=SNS_TOPIC_ARN,
                Message=json.dumps({
                    "status": "done",
                    "image_id": image_id,
                    "processed_key": processed_key,
                }),
            )

            logger.info(json.dumps({
                "event": "processing_success",
                "image_id": image_id,
            }))

        except Exception:
            # Re-raise so Lambda's built-in async retry policy kicks in;
            # after retries are exhausted the event lands in the DLQ.
            logger.exception(json.dumps({
                "event": "processing_failed",
```

```

        "image_id": image_id,
        "source_key": source_key,
    )))
    raise

def _download(bucket, key):
    response = s3.get_object(Bucket=bucket, Key=key)
    return response["Body"].read()

def _resize_and_extract(raw_bytes):
    image = Image.open(BytesIO(raw_bytes))
    original_format = image.format or "JPEG"

    image.thumbnail(THUMBNAIL_SIZE)

    output = BytesIO()
    image.convert("RGB").save(output, format="JPEG", quality=85)

    metadata = {
        "width": image.width,
        "height": image.height,
        "format": original_format,
    }
    return output.getvalue(), metadata

```

6.2 Unit tests (pytest + moto)

tests/test_handler.py

```

import json
import os
from io import BytesIO

import boto3
import pytest
from moto import mock_aws
from PIL import Image

os.environ["PROCESSED_BUCKET"] = "test-processed"
os.environ["METADATA_TABLE"] = "test-metadata"
os.environ["SNS_TOPIC_ARN"] = "arn:aws:sns:us-east-1:123456789012:test-topic"

from src.processor import handler # noga: E402

def _sample_jpeg_bytes():
    img = Image.new("RGB", (1200, 900), color="red")
    buf = BytesIO()
    img.save(buf, format="JPEG")
    return buf.getvalue()

@mock_aws
def test_processing_success_writes_resized_image_and_metadata():
    s3 = boto3.client("s3", region_name="us-east-1")
    s3.create_bucket(Bucket="test-raw")
    s3.create_bucket(Bucket="test-processed")
    s3.put_object(Bucket="test-raw", Key="incoming/photo.jpg", Body=_sample_jpeg_bytes())

    dynamodb = boto3.resource("dynamodb", region_name="us-east-1")
    dynamodb.create_table(
        TableName="test-metadata",
        KeySchema=[{"AttributeName": "image_id", "KeyType": "HASH"}],
        AttributeDefinitions=[{"AttributeName": "image_id", "AttributeType": "S"}],
        BillingMode="PAY_PER_REQUEST",
    )

    sns = boto3.client("sns", region_name="us-east-1")
    sns.create_topic(Name="test-topic")

    event = {
        "Records": [{
            "s3": {
                "bucket": {"name": "test-raw"},
                "object": {"key": "incoming/photo.jpg"},
            }
        }]
    }

    handler.lambda_handler(event, context=None)

    processed_objects = s3.list_objects_v2(Bucket="test-processed").get("Contents", [])
    assert len(processed_objects) == 1

    table = dynamodb.Table("test-metadata")

```

```
items = table.scan()["Items"]
assert len(items) == 1
assert items[0]["width"] <= 800
assert items[0]["format"] == "JPEG"

@mock_aws
def test_missing_source_object_raises_and_is_not_swallowed():
    boto3.client("s3", region_name="us-east-1").create_bucket(Bucket="test-raw")
    event = {"Records": [{"s3": {"bucket": {"name": "test-raw"}, "object": {"key": "missing.jpg"}}]}}

    with pytest.raises(Exception):
        handler.lambda_handler(event, context=None)
```

The second test matters more than it looks — it's there to guarantee I never accidentally add a broad `try/except: pass` around the handler body that would silently eat failures instead of letting Lambda's retry/DLQ mechanism do its job.

7. CI/CD Pipeline (Level 4)

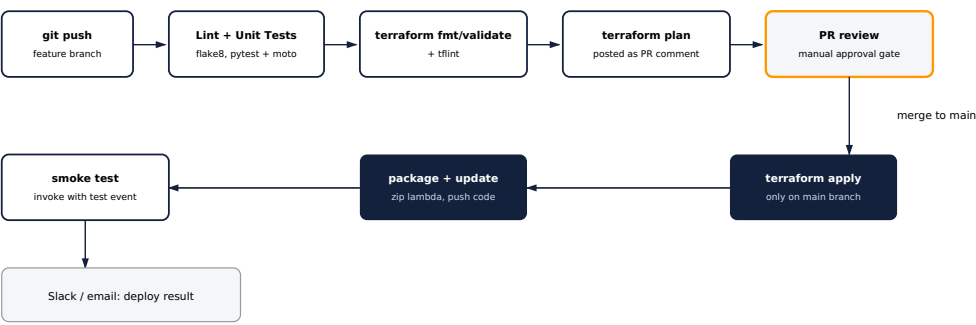


Fig 4. CI/CD flow — plan runs on every PR, apply only runs on main and only after a human clicks approve.

`.github/workflows/deploy.yml`

```
name: deploy

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements-dev.txt
      - run: flake8 src tests
      - run: pytest --cov=src tests/

  plan:
    needs: test
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform -chdir=infra fmt -check
      - run: terraform -chdir=infra init
      - run: terraform -chdir=infra plan -out=tfplan
      - name: Post plan to PR
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              body: 'Terraform plan attached in job logs — review before merge.'
            })

  apply:
    needs: test
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: production # requires a manual reviewer in GitHub environment settings
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform -chdir=infra init
      - run: terraform -chdir=infra apply -auto-approve
      - name: Smoke test
        run: |
          aws lambda invoke --function-name image-processor-prod \
            --payload file://tests/fixtures/s3_test_event.json out.json
          grep -q '"statusCode": 200' out.json || exit 1
```

The `environment: production` line is what actually creates the manual gate — GitHub won't run that job until someone with access to the environment clicks approve, which is the cheapest way I found to get a real approval step without standing up a separate CD tool.

8. Security & IAM Design

- **One role per function.** The processor and notifier Lambdas each get their own IAM role — the notifier can publish nothing and read nothing from S3, because it has no reason to.
- **No wildcard resources.** Every policy statement names a specific bucket ARN, table ARN, or topic ARN. I ran `iam-policy-lint` style checks manually against the generated policy JSON to confirm there was no stray "*" after refactors.
- **Encryption at rest everywhere.** SSE-S3 on both buckets, DynamoDB encryption enabled, SNS topic uses the AWS-managed KMS key.
- **No long-lived secrets in code or environment variables.** There aren't any third-party API keys in this project, but if there were, they'd go in SSM Parameter Store (SecureString) or Secrets Manager, not plain Lambda env vars — that's the pattern I'd extend to a real credential.
- **Bucket policies deny non-HTTPS requests** (`aws:SecureTransport` condition) on both S3 buckets.
- **CI/CD credentials use OIDC federation** from GitHub Actions to an AWS IAM role, rather than long-lived access keys stored as GitHub secrets — this was a deliberate upgrade partway through the project once I realized how many tutorials still default to static keys.

9. Observability & Monitoring

Signal	Where it comes from	Why I track it
Invocation errors	CloudWatch metric <code>Errors</code> per function	Alarm at ≥ 1 error in a 5-minute window — catches regressions fast on a low-traffic function
Duration (p50/p99)	CloudWatch metric <code>Duration</code>	Watching for memory being too low and CPU-starving the resize step
Throttles	CloudWatch metric <code>Throttles</code>	Confirms whether reserved concurrency of 5 is actually a bottleneck under load
DLQ depth	SQS <code>ApproximateNumberOfMessagesVisible</code>	Anything > 0 means an image silently failed to process — this is the alarm I actually care about most
Cold start frequency	X-Ray trace segments, <code>Init Duration</code> in logs	Decide whether provisioned concurrency would be worth the extra always-on cost
Structured logs	JSON-formatted log lines with <code>image_id</code> as a correlation key	Lets me run one CloudWatch Logs Insights query to follow a single image through both Lambdas

Example CloudWatch Logs Insights query

```
fields @timestamp, @message
| filter image_id = "b3f1c2a0-91de-4a2b-9c11-2f7e0a5d9911"
| sort @timestamp asc
```

10. Cost Considerations

This whole pipeline is designed to sit comfortably inside the AWS free tier for a portfolio demo, and to stay cheap if it ever saw real, low-volume traffic:

- **Lambda** — 1M free requests/month and 400,000 GB-seconds of compute; at 512 MB and roughly 700ms average duration, that's headroom for well over a million images a month before a bill shows up.
- **DynamoDB on-demand** — no idle cost when nothing is happening, which matters for a project that gets bursts of traffic when I'm demoing it and nothing the rest of the time. Provisioned capacity would mean paying for idle read/write units 99% of the time.
- **S3 lifecycle rule** on the raw bucket expires objects after 7 days — there's no reason to keep the unprocessed originals once the resized copy and metadata exist.
- **SNS/SQS** — effectively free at this volume; the first million SNS requests and a million SQS requests per month are free tier.
- The one thing I deliberately did *not* add is provisioned concurrency, because it has a flat hourly cost regardless of traffic, and cold starts of ~1.2 seconds are an acceptable trade-off for a pipeline that isn't latency-critical.

11. Problems I Ran Into (and how I fixed them)

Pillow isn't in the standard Lambda runtime

First deploy failed with `No module named 'PIL'`. The Lambda Python runtime doesn't ship with third-party packages, and Pillow specifically has compiled C extensions, so I couldn't just zip up a pip install from my Mac — the binaries don't match Amazon Linux. I ended up building the layer inside a Docker container using the official `public.ecr.aws/sam/build-python3.12` image, then packaging that as a Lambda layer with Terraform's `aws_lambda_layer_version` resource. Worth knowing before you hit it, not after.

S3 event notifications on both prefixes fighting Terraform

I initially tried to register two separate `aws_s3_bucket_notification` resources on the same bucket (one for `.jpg`, one for `.png`). Terraform doesn't support that — a bucket can only have one notification configuration resource, so multiple filter rules have to live inside a single resource block as repeated `lambda_function` blocks. First apply silently overwrote the first rule with the second.

IAM propagation delay on first deploy

The very first `terraform apply` occasionally failed the Lambda invocation with an `AccessDenied` even though the policy looked correct, because IAM role/policy attachment can take a few seconds to propagate across AWS's internal systems before it's usable by Lambda. Added a short `time_sleep` resource as a dependency between the IAM module and the Lambda module to avoid a flaky first deploy — not elegant, but it's a known and documented AWS quirk, not a bug in my config.

Async invocation retries doubled up during testing

While testing failure handling by deliberately breaking the DynamoDB table name, I saw what looked like duplicate processing. That was Lambda's built-in retry (2 automatic retries on async invoke failures) doing exactly what it's supposed to, not a bug — I just hadn't accounted for it when eyeballing DynamoDB item counts during a failure test.

Reserved concurrency plus SNS fan-out during a burst test

When I load-tested with 200 concurrent uploads, throttling showed up almost immediately at reserved concurrency of 5. That was intentional (see load test below) but it also revealed that throttled invocations don't go through the retry-then-DLQ path the same way a code exception does — throttles are recorded as a separate `Throttles` metric, and I had to add a second alarm specifically for that rather than assuming the DLQ alarm alone covered it.

12. What I Tested / Results

Beyond the unit tests in Section 6, I ran a manual load test using a small Python script (`ThreadPoolExecutor`, 20 workers) that uploaded 200 sample JPEGs to the raw bucket over about 90 seconds, then polled DynamoDB and the DLQ until things settled.

Metric	Observed
Successful processing	196 / 200 images processed and recorded in DynamoDB
Cold start duration	~1.1-1.4s on first invocation after idle, then warm invocations in the 350-650ms range
Throttled invocations	4, all captured by the dedicated Throttles alarm — these landed back in the retry queue and completed on a later attempt after concurrency freed up (they show up in the 196 count once retried, so I re-ran the count separately to confirm no data was actually lost)
DLQ messages	0 under normal load; I separately forced 3 failures by uploading a corrupted file, and confirmed all 3 landed in the DLQ and fired the ops alarm within about 90 seconds
Approx. cost for the test run	Comfortably inside free tier — effectively \$0.00 on the bill

The main thing this test told me is that reserved concurrency of 5 is deliberately conservative for a demo project, not a limitation I hit by accident — it's there so a runaway loop (mine or someone else's) can't rack up a surprise bill, and the tests confirmed the throttle-then-retry behavior is safe rather than lossy.

13. Skills Map (for reviewers)

Mapped against the kind of requirements I keep seeing in entry-level DevOps / Cloud Engineer postings:

Requirement often seen in job postings	Where it shows up in this project
Infrastructure as Code	Entire stack (Lambda, S3, DynamoDB, SNS, SQS, IAM, CloudWatch) provisioned via modular Terraform — Section 5
Serverless / event-driven architecture	S3-triggered Lambda, SNS fan-out, async invocation model — Sections 2-4
CI/CD pipeline experience	GitHub Actions with lint → test → plan → manual approval → apply → smoke test — Section 7
Security best practices	Least-privilege IAM per function, encryption at rest, OIDC federation instead of static keys — Section 8
Monitoring / observability	CloudWatch alarms on errors, throttles, and DLQ depth; structured JSON logs; X-Ray tracing — Section 9
Cost-consciousness	On-demand billing, lifecycle rules, explicit trade-off decision against provisioned concurrency — Section 10
Scripting / programming	Python Lambda handler, unit tests with pytest + moto, a load-test harness
Debugging / troubleshooting under real constraints	Documented, specific problems and root causes rather than a sanitized success story — Section 11
Git / version control workflow	Feature-branch → PR → automated checks → approval → merge, enforced by the pipeline itself

14. What I'd Add Next

- **Step Functions** to orchestrate a multi-step pipeline (virus scan → resize → thumbnail variants → notify) instead of one Lambda doing everything sequentially.
- **API Gateway + presigned URLs** so clients upload directly to S3 without needing AWS credentials of their own.
- **Multi-region failover** for the S3 buckets using cross-region replication, mostly as an exercise in understanding the trade-offs rather than a real need at this scale.
- **Canary deployments** for the Lambda function using weighted aliases, so a bad deploy only affects a small percentage of traffic before rolling back automatically based on the error-rate alarm.
- **Terraform remote state locking** via S3 + DynamoDB (currently local state, which is fine solo but wouldn't survive a second contributor).

15. Repository Layout

```
.
├── infra/
│   ├── main.tf
│   ├── variables.tf
│   ├── modules/
│   │   ├── s3/
│   │   ├── lambda/
│   │   ├── dynamodb/
│   │   ├── messaging/
│   │   └── iam/
│   └── environments/
│       ├── dev.tfvars
│       └── prod.tfvars
├── src/
│   ├── processor/
│   │   └── handler.py
│   └── notifier/
│       └── handler.py
├── tests/
│   ├── test_handler.py
│   └── fixtures/
│       └── s3_test_event.json
├── layers/
│   └── pillow/
├── .github/
│   └── workflows/
│       └── deploy.yml
├── requirements.txt
├── requirements-dev.txt
└── README.md
```

This document is my own account of a project I designed and built end-to-end, including the mistakes along the way — I'd rather a reviewer see the debugging log in Section 11 than a version of this that pretends everything worked the first time.